

finance.c

```
#include <stdio.h>
#include "types.h"
#include "functions.h"

/* Sets a square back to "unowned": no owner, no mortgage, no loan
pledge and no insurance. Used when selling, foreclosing or
liquidating a property. */
void stripOwnership(GameState game[], int index)
{
    game[0].board[index].property.owner = -1;
    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;
    game[0].board[index].property.insurance = NO_INSURANCE;
}

/* Clears a player's loan completely (amount, rate and rounds). */
void resetLoan(GameState game[], int playerIndex)
{
    game[0].players[playerIndex].loan.active = 0;
    game[0].players[playerIndex].loan.amount = 0;
    game[0].players[playerIndex].loan.interestRate = 0;
    game[0].players[playerIndex].loan.remainingRounds = 0;
}

/* Adds delta to the owned-count that matches the square's type
(property, railway or utility). Pass 1 when buying, -1 when
selling or losing a square. */
void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)
{
    if(game[0].board[index].type == PROPERTY)
        game[0].players[playerIndex].propertiesOwned += delta;
    else if(game[0].board[index].type == RAILWAY)
        game[0].players[playerIndex].railwaysOwned += delta;
    else if(game[0].board[index].type == UTILITY)
        game[0].players[playerIndex].utilitiesOwned += delta;
}

/* The colour group of a square, or -1 if it isn't a property. */
PropertyGroup groupOf(GameState game[], int index)
{
    if(game[0].board[index].type == PROPERTY)
        return game[0].board[index].property.group;

    return (PropertyGroup)-1;
}

void receiveMoney(GameState game[], int playerIndex, int amount)
{
    /* bankrupt players take no further part in the game (Section 1.3) */
    if(game[0].players[playerIndex].bankrupt)
        return;

    game[0].players[playerIndex].cash += amount;
}

void payMoney(GameState game[], int playerIndex, int amount)
{
    game[0].players[playerIndex].cash -= amount;

    /* Section 3.4: a Risk Taker facing bankruptcy first sells their
```

```

    cheapest undeveloped property to the Bank to raise cash.
    Loan-locked collateral (Rule-LK 3) can't be sold, so stop as
    soon as a sale fails to reduce the stock of undeveloped
    properties - otherwise this loop would never end. */
while(game[0].players[playerIndex].cash < 0 &&
    !game[0].players[playerIndex].bankrupt &&
    game[0].players[playerIndex].strategy == RISK_TAKER)
{
    int before = countUndevelopedProperties(game, playerIndex);

    sellLowValueProperty(game, playerIndex);

    if(countUndevelopedProperties(game, playerIndex) >= before)
        break;
}

if(game[0].players[playerIndex].cash < 0 && !game[0].players[playerIndex].bankrupt)
{
    game[0].players[playerIndex].bankrupt = 1;

    printf("\n*** BANKRUPTCY ***\n");
    printf("%s has been declared bankrupt.\n",
        game[0].players[playerIndex].name);

    liquidateBankruptAssets(game, playerIndex);

    /* cash never stays negative - assets were liquidated, so any
       remaining debt is written off (Section 1.3) */
    game[0].players[playerIndex].cash = 0;
}
}

/* Rule 14: on bankruptcy, buildings are demolished and every
   property/railway/utility owned is auctioned off (Rule-LK 19). */
void liquidateBankruptAssets(GameState game[], int playerIndex)
{
    int i;

    printf("%s's remaining assets are being liquidated.\n",
        game[0].players[playerIndex].name);

    resetLoan(game, playerIndex);

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        demolishBuildingsOn(game, i);

        stripOwnership(game, i);
        game[0].board[i].property.damaged = 0;
        game[0].board[i].property.repairCostOwed = 0;

        adjustOwnedCount(game, playerIndex, i, -1);

        runAuction(game, i, -1);
    }
}

/* Income Tax (Rule-LK): 15% of the player's net worth at the base
   rate. Market conditions scale the rate (currentTaxRatePercent). */
void payTax(GameState game[], int playerIndex)

```

```

{
    int rate;
    int netWorth;
    int amount;

    rate = currentTaxRatePercent(game, INCOME_TAX_RATE);
    netWorth = calculateNetWorth(game, playerIndex);
    amount = (netWorth * rate) / 100;

    /* a bankrupt player's net worth can be negative - a tax can never
       pay the player money, so clamp it at zero */
    if(amount < 0)
        amount = 0;

    printf("\n%s landed on Income Tax.\n",
           game[0].players[playerIndex].name);

    printf("Tax Rate : %d%%\n", rate);
    printf("Net Worth : LKR %d\n", netWorth);
    printf("%s paid tax : LKR %d\n",
           game[0].players[playerIndex].name,
           amount);

    payMoney(game, playerIndex, amount);
}

/* Community Development Fund (Rule-LK): 10% of the player's total
   assets at current market rate. Only property (land) counts -
   buildings are not included. The 10% rate is scaled by market
   conditions just like the Income Tax rate. */
void payCommunityFundTax(GameState game[], int playerIndex)
{
    int rate;
    int assets;
    int amount;

    rate = currentTaxRatePercent(game, COMMUNITY_FUND_TAX_RATE);
    assets = calculatePropertyValue(game, playerIndex);
    amount = (assets * rate) / 100;

    printf("\n%s landed on Community Development Fund.\n",
           game[0].players[playerIndex].name);

    printf("Tax Rate : %d%%\n", rate);
    printf("Property Assets (market rate) : LKR %d\n", assets);
    printf("%s paid community development tax : LKR %d\n",
           game[0].players[playerIndex].name,
           amount);

    payMoney(game, playerIndex, amount);
}

int findPropertyToMortgage(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.mortgaged)
            continue;
    }
}

```

```

        if(game[0].board[i].property.loanLocked)
            continue;

        /* must be undeveloped before it can be mortgaged */
        if(game[0].board[i].type == PROPERTY &&
            (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
        {
            continue;
        }

        return i;
    }

    return -1;
}

int findMortgagedProperty(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner == playerIndex &&
            game[0].board[i].property.mortgaged)
        {
            return i;
        }
    }

    return -1;
}

void mortgageProperty(GameState game[], int playerIndex)
{
    int propIndex;
    int payout;
    PropertyGroup group;

    if(!shouldMortgage(game, playerIndex))
        return;

    propIndex = findPropertyToMortgage(game, playerIndex);

    if(propIndex == -1)
        return;

    /* Rule-LK 34: a booming group's mortgage values rise by 15%, a
       declining group's fall by 10%. Railways and utilities are
       unaffected. */
    group = groupOf(game, propIndex);

    payout = (game[0].board[propIndex].property.mortgageValue *
        modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1)) / 100;

    game[0].board[propIndex].property.mortgaged = 1;

    receiveMoney(game, playerIndex, payout);

    printf("\n%s mortgaged %s for LKR %d.\n",
        game[0].players[playerIndex].name,
        game[0].board[propIndex].name,
        payout);
}

```

```

}

void redeemMortgage(GameState game[], int playerIndex)
{
    int propIndex;
    int redeemCost;

    propIndex = findMortgagedProperty(game, playerIndex);

    if(propIndex == -1)
        return;

    /* pay back the mortgage value plus 10% interest */
    redeemCost = (game[0].board[propIndex].property.mortgageValue * MORTGAGE_REPAY_PERCENT) / 100;

    if(!shouldRedeemMortgage(game, playerIndex, redeemCost))
        return;

    if(game[0].players[playerIndex].cash < redeemCost)
        return;

    payMoney(game, playerIndex, redeemCost);

    game[0].board[propIndex].property.mortgaged = 0;

    printf("\n%s redeemed the mortgage on %s for LKR %d.\n",
        game[0].players[playerIndex].name,
        game[0].board[propIndex].name,
        redeemCost);
}

void handleMortgageDecisions(GameState game[], int playerIndex)
{
    redeemMortgage(game, playerIndex);
    mortgageProperty(game, playerIndex);
}

/* Rent for a normal property, based on houses/hotel (Table 6),
   scaled by any active bonuses and by building condition (Table 3). */
int calculateRent(GameState game[], int position)
{
    int rent;
    int conditionPercent;
    int houses;
    int hotel;
    int baseRent;
    PropertyGroup group;
    int condition;

    houses = game[0].board[position].property.houses;
    hotel = game[0].board[position].property.hotel;
    baseRent = game[0].board[position].property.baseRent;
    group = game[0].board[position].property.group;
    condition = game[0].board[position].property.condition;

    if(hotel)
    {
        rent = baseRent * 10;
    }
    else
    {
        if(houses == 0)
            rent = baseRent * 1;
    }
}

```

```

        else if(houses == 1)
            rent = baseRent * 2;
        else if(houses == 2)
            rent = baseRent * 3;
        else if(houses == 3)
            rent = baseRent * 5;
        else
            rent = baseRent * 7;
    }

    /* Rule-LK 34: all matching modifiers multiply together. */
    rent = (rent * modifierMultiplier(game, MOD_GROUP_RENT, group, -1)) / 100;
    rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

    if(hotel)
        rent = (rent * modifierMultiplier(game, MOD_HOTEL_RENT, -1, -1)) / 100;

    if(houses > 0 || hotel)
    {
        conditionPercent = rentConditionPercent(condition);
        rent = (rent * conditionPercent) / 100;
    }

    return rent;
}

/* Rent for a railway, based on how many the same owner has (Table 7). */
int calculateRailwayRent(GameState game[], int playerIndex)
{
    int i;
    int count;
    int baseRent;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == RAILWAY &&
            game[0].board[i].property.owner == playerIndex)
        {
            count++;
        }
    }

    if(count == 1)
        baseRent = RAILWAY_RENT_1;
    else if(count == 2)
        baseRent = RAILWAY_RENT_2;
    else if(count == 3)
        baseRent = RAILWAY_RENT_3;
    else if(count == 4)
        baseRent = RAILWAY_RENT_4;
    else
        baseRent = 0;

    baseRent = (baseRent * modifierMultiplier(game, MOD_RAIL_RENT, -1, -1)) / 100;
    baseRent = (baseRent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

    return baseRent;
}

/* Rent for a utility, based on the dice roll just made (Table 8). */
int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)

```

```

{
    int i;
    int count;
    int rent;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == UTILITY &&
           game[0].board[i].property.owner == playerIndex)
        {
            count++;
        }
    }

    if(count == 2)
        rent = UTILITY_RENT_TWO * diceValue;
    else if(count == 1)
        rent = UTILITY_RENT_ONE * diceValue;
    else
        rent = 0;

    rent = (rent * modifierMultiplier(game, MOD_UTIL_RENT, -1, -1)) / 100;
    rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

    return rent;
}

int groupSize(GameState game[], PropertyGroup group)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
            count++;
    }

    return count;
}

int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)
{
    int i;
    int owned;

    owned = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
        {
            if(game[0].board[i].property.owner == playerIndex)
                owned++;
        }
    }

    return (owned == groupSize(game, group));
}

```

```

/* Builds one house (or upgrades to a hotel) somewhere in this
   colour group. Always adds to whichever property currently has the
   fewest houses, so development stays even (Rule 9). Returns 1 if
   something was actually built, 0 if not - constructBuildings()
   uses this to keep building repeatedly in the same turn. */
int developGroup(GameState game[], int playerIndex, PropertyGroup group)
{
    int i;
    int minHouses;
    int targetIndex;
    int allFourHouses;

    if(!ownsMonopoly(game, playerIndex, group))
        return 0;

    minHouses = MAX_HOUSES + 1;
    targetIndex = -1;
    allFourHouses = 1;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
        {
            /* can't build on a mortgaged, damaged or loan-locked
               property - those earn no rent and/or are pledged */
            if(game[0].board[i].property.mortgaged ||
               game[0].board[i].property.damaged ||
               game[0].board[i].property.loanLocked)
            {
                /* Rule 9: a blocked member that isn't fully developed
                   means the group can't support a hotel yet */
                if(!game[0].board[i].property.hotel &&
                    game[0].board[i].property.houses < MAX_HOUSES)
                {
                    allFourHouses = 0;
                }

                continue;
            }

            if(game[0].board[i].property.hotel)
                continue;

            if(game[0].board[i].property.houses < MAX_HOUSES)
                allFourHouses = 0;

            if(game[0].board[i].property.houses < minHouses)
            {
                minHouses = game[0].board[i].property.houses;
                targetIndex = i;
            }
        }
    }

    if(targetIndex == -1)
        return 0; /* every property already has a hotel */

    /* Rule 9: never pile a 5th house onto one property just because a
       group member is blocked from building */
    if(game[0].board[targetIndex].property.houses >= MAX_HOUSES)
        return 0;

```

```

if(allFourHouses)
{
    int hotelCost;
    char costBuf[32];

    /* Rule-LK 13/31/34: base hotel cost is permanent; temporary
       events only add modifiers on top. A market boom's +10%
       applies only to the booming group, while event/regulation
       modifiers (group -1) are global. */
    hotelCost = (game[0].board[targetIndex].property.hotelCost *
                 modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;

    if(!shouldConstruct(game, playerIndex, hotelCost, 1))
        return 0;

    if(game[0].players[playerIndex].cash < hotelCost)
        return 0;

    payMoney(game, playerIndex, hotelCost);

    game[0].board[targetIndex].property.houses = 0;
    game[0].board[targetIndex].property.hotel = 1;

    printf("\n%s upgraded %s to a Hotel.\n",
           game[0].players[playerIndex].name,
           game[0].board[targetIndex].name);

    formatLKR(hotelCost, costBuf);
    printf("Construction Cost : LKR %s.\n", costBuf);

    return 1;
}

{
    int houseCost;
    char costBuf[32];

    houseCost = (game[0].board[targetIndex].property.houseCost *
                 modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;

    if(!shouldConstruct(game, playerIndex, houseCost, 0))
        return 0;

    if(game[0].players[playerIndex].cash < houseCost)
        return 0;

    payMoney(game, playerIndex, houseCost);

    game[0].board[targetIndex].property.houses++;

    printf("\n%s constructed one house on %s.\n",
           game[0].players[playerIndex].name,
           game[0].board[targetIndex].name);

    formatLKR(houseCost, costBuf);
    printf("Construction Cost : LKR %s.\n", costBuf);

    return 1;
}

}

/* Keeps building on each colour group, one step at a time, until
   either nothing more can be built there this turn (unaffordable,

```

```

the strategy doesn't want to, or the group is fully built up) -
this is what lets a strategy build "the maximum possible number
of houses immediately" in a single turn instead of one per turn. */
void constructBuildings(GameState game[], int playerIndex)
{
    PropertyGroup group;
    int builtSomething;
    int safetyLimit;

    if(game[0].economy.constructionSuspendedRoundsLeft > 0)
        return;

    for(group = BROWN; group < NO_GROUP; group++)
    {
        safetyLimit = 0;

        do
        {
            builtSomething = developGroup(game, playerIndex, group);
            safetyLimit++;
        }
        while(builtSomething && safetyLimit < 20);
    }
}

int calculatePropertyValue(GameState game[], int playerIndex)
{
    int i;
    int total;

    total = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        /* mortgaged properties contribute zero to net worth */
        if(game[0].board[i].property.mortgaged)
            continue;

        /* railways and utilities also use currentMarketValue so their
           value modifiers (rail/port/water effects) apply too */
        total += currentMarketValue(game, i);
    }

    return total;
}

int calculateBuildingValue(GameState game[], int playerIndex)
{
    int i;
    int total;

    total = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;
    }
}

```

```

        if(game[0].board[i].property.hotel)
            total += game[0].board[i].property.hotelCost;
        else
            total += game[0].board[i].property.houses * game[0].board[i].property.houseCost;
    }

    return total;
}

/* Rule 15 net worth formula: cash + property value + building value,
   minus any outstanding loan amount. */
int calculateNetWorth(GameState game[], int playerIndex)
{
    int cash;
    int propertyValue;
    int buildingValue;
    int outstandingLoans;

    cash = game[0].players[playerIndex].cash;
    propertyValue = calculatePropertyValue(game, playerIndex);
    buildingValue = calculateBuildingValue(game, playerIndex);

    outstandingLoans = 0;

    if(game[0].players[playerIndex].loan.active)
        outstandingLoans = game[0].players[playerIndex].loan.amount;

    return cash + propertyValue + buildingValue - outstandingLoans;
}

int countUndevelopedProperties(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
           game[0].board[i].property.owner == playerIndex &&
           game[0].board[i].property.houses == 0 &&
           !game[0].board[i].property.hotel)
        {
            count++;
        }
    }

    return count;
}

/* True if buying the property at pos would complete the whole colour
   group for the player (Section 3.1). */
int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)
{
    int i;
    PropertyGroup group;

    if(game[0].board[pos].type != PROPERTY)
        return 0;

    if(game[0].board[pos].property.owner != -1)

```

```

        return 0;

group = game[0].board[pos].property.group;

for(i = 0; i < BOARD_SIZE; i++)
{
    if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
    {
        if(i != pos && game[0].board[i].property.owner != playerIndex)
            return 0;
    }
}

return 1;
}

/* Sells the player's lowest-value undeveloped property back to the
   Bank at market value (Section 3.4 bankruptcy tactic). */
void sellLowValueProperty(GameState game[], int playerIndex)
{
    int i;
    int best;
    int bestValue;
    int value;

    best = -1;
    bestValue = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
            continue;

        /* Rule-LK 3: loan-locked collateral can never be sold */
        if(game[0].board[i].property.loanLocked)
            continue;

        value = currentMarketValue(game, i);

        if(best == -1 || value < bestValue)
        {
            best = i;
            bestValue = value;
        }
    }

    if(best == -1)
        return;

    receiveMoney(game, playerIndex, bestValue);

    stripOwnership(game, best);

    adjustOwnedCount(game, playerIndex, best, -1);

    printf("\n%s sold %s to the Bank for LKR %d.\n",
        game[0].players[playerIndex].name,

```

```

        game[0].board[best].name,
        bestValue);
    }

/* Section 3.4: the Opportunistic Trader sells properties expected to
lose value following economic events - any undeveloped property in
a group that is currently declining, under a global downturn, or
carrying its own negative value modifier. Loan-locked collateral
(Rule-LK 3) is never sold. */
void sellDecliningProperties(GameState game[], int playerIndex)
{
    int i;
    int globalMult;

    if(game[0].players[playerIndex].strategy != OPPORTUNISTIC_TRADER)
        return;

    if(game[0].players[playerIndex].bankrupt)
        return;

    globalMult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);

    for(i = 0; i < BOARD_SIZE; i++)
    {
        int groupMult;
        int indexMult;
        int value;

        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
            continue;

        if(game[0].board[i].property.loanLocked)
            continue;

        groupMult = modifierMultiplier(game, MOD_GROUP_VALUE,
                                       game[0].board[i].property.group, -1);
        indexMult = modifierMultiplier(game, MOD_INDEX_VALUE, -1, i);

        if(globalMult >= 100 && groupMult >= 100 && indexMult >= 100)
            continue;

        value = currentMarketValue(game, i);

        receiveMoney(game, playerIndex, value);

        stripOwnership(game, i);

        adjustOwnedCount(game, playerIndex, i, -1);

        printf("\n%s sold %s to the Bank for LKR %d "
               "(expected to lose value).\n",
               game[0].players[playerIndex].name,
               game[0].board[i].name,
               value);
    }
}

```

```

/* Releases the player's lowest-value undeveloped property and puts
   it up for auction (Anti-Speculation Act enforcement). */
void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)
{
    int i;
    int best;
    int bestValue;
    int value;

    best = -1;
    bestValue = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
            continue;

        /* Rule-LK 3: loan-locked collateral can never be auctioned */
        if(game[0].board[i].property.loanLocked)
            continue;

        value = currentMarketValue(game, i);

        if(best == -1 || value < bestValue)
        {
            best = i;
            bestValue = value;
        }
    }

    if(best == -1)
        return;

    printf("\nAnti-Speculation Act : %s must sell %s.\n",
           game[0].players[playerIndex].name,
           game[0].board[best].name);

    stripOwnership(game, best);

    adjustOwnedCount(game, playerIndex, best, -1);

    runAuction(game, best, playerIndex);
}

/* Sells excess undeveloped properties until the player is back at
   the allowed limit (or nothing more can be sold). */
void forceSellExcess(GameState game[], int playerIndex)
{
    int excess;

    excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;

    while(excess > 0)
    {
        int before = countUndevelopedProperties(game, playerIndex);

        sellUndevelopedPropertyToAuction(game, playerIndex);
    }
}

```

```

        /* nothing sellable left (e.g. every excess property is
           loan-locked) - stop to avoid a non-terminating loop */
        if(countUndevelopedProperties(game, playerIndex) >= before)
            break;

        excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;
    }
}

/* Rule-LK 8 (Anti-Speculation Act): once active, a player may keep
   at most three undeveloped properties. After 5 consecutive rounds
   above the limit the excess is force-auctioned. */
void enforceAntiSpeculation(GameState game[])
{
    int i;
    int undeveloped;

    if(!game[0].economy.antiSpeculationActive)
    {
        for(i = 0; i < MAX_PLAYERS; i++)
            game[0].players[i].antiSpecRounds = 0;

        return;
    }

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
        {
            game[0].players[i].antiSpecRounds = 0;
            continue;
        }

        undeveloped = countUndevelopedProperties(game, i);

        if(undeveloped > MAX_UNDEVELOPED_PROPS)
        {
            game[0].players[i].antiSpecRounds++;

            if(game[0].players[i].antiSpecRounds >= ANTI_SPEC_TRIGGER_ROUNDS)
            {
                forceSellExcess(game, i);
                game[0].players[i].antiSpecRounds = 0;
            }
        }
        else
        {
            game[0].players[i].antiSpecRounds = 0;
        }
    }
}

void buyProperty(GameState game[], int playerIndex)
{
    int pos;
    int price;
    int wantsToBuy;
    char priceBuf[32];
    char balanceBuf[32];

    pos = game[0].players[playerIndex].position;

```

```

if(game[0].board[pos].type != PROPERTY &&
    game[0].board[pos].type != RAILWAY &&
    game[0].board[pos].type != UTILITY)
{
    return;
}

if(game[0].board[pos].property.owner != -1)
    return; /* already owned */

price = currentMarketValue(game, pos);

if(game[0].board[pos].type == PROPERTY)
{
    /* Rule-LK 31/34: a booming group's direct purchase prices
       rise by 15% (group-scoped, not global). Railways and
       utilities are priced at their current market value, so
       their value modifiers (MOD_RAIL_VALUE, MOD_INDEX_VALUE)
       affect their purchase price too. */
    price = (price *
              modifierMultiplier(game, MOD_PURCHASE_PRICE,
                                  game[0].board[pos].property.group, -1)) / 100;
}

wantsToBuy = shouldBuyProperty(game, playerIndex);

if(game[0].players[playerIndex].cash < price)
    wantsToBuy = 0;

/* Rule 5: if they don't buy directly, it goes to auction */
if(!wantsToBuy)
{
    runAuction(game, pos, -1);
    return;
}

payMoney(game, playerIndex, price);

game[0].board[pos].property.owner = playerIndex;

if(game[0].board[pos].type == PROPERTY)
    game[0].players[playerIndex].propertiesOwned++;
else if(game[0].board[pos].type == RAILWAY)
    game[0].players[playerIndex].railwaysOwned++;
else if(game[0].board[pos].type == UTILITY)
    game[0].players[playerIndex].utilitiesOwned++;

formatLKR(price, priceBuf);
formatLKR(game[0].players[playerIndex].cash, balanceBuf);

printf("\n%s purchased %s for LKR %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[pos].name,
        priceBuf);

printf("Remaining Balance : LKR %s.\n",
        balanceBuf);
}

void payRent(GameState game[], int playerIndex, int diceValue)
{
    int pos;
    int owner;

```

```

int rent;
char rentBuf[32];

pos = game[0].players[playerIndex].position;

if(game[0].board[pos].type != PROPERTY &&
    game[0].board[pos].type != RAILWAY &&
    game[0].board[pos].type != UTILITY)
{
    return;
}

owner = game[0].board[pos].property.owner;

if(owner == playerIndex)
{
    /* landed on your own square - no rent, maybe renovate */
    if(game[0].board[pos].type == PROPERTY)
        tryRenovateAgeDepreciation(game, playerIndex, pos);
    return;
}

if(owner == -1)
    return;

if(game[0].board[pos].property.mortgaged)
    return;

if(pos == game[0].economy.closedPropertyIndex && game[0].economy.closedPropertyRoundsLeft > 0)
{
    printf("\n%s landed on %s, but it is closed (Political Rally) - no rent.\n",
        game[0].players[playerIndex].name, game[0].board[pos].name);
    return;
}

if(game[0].board[pos].type == PROPERTY && game[0].board[pos].property.damaged)
{
    printf("\n%s landed on %s, but it is damaged and collects no rent.\n",
        game[0].players[playerIndex].name, game[0].board[pos].name);
    return;
}

if(game[0].board[pos].type == PROPERTY &&
    game[0].board[pos].property.lostIncomeRoundsLeft > 0)
{
    printf("\n%s landed on %s, but it earns no rent right now "
        "(Business Interruption).\n",
        game[0].players[playerIndex].name, game[0].board[pos].name);
    return;
}

if(game[0].board[pos].type == PROPERTY)
    rent = calculateRent(game, pos);
else if(game[0].board[pos].type == RAILWAY)
    rent = calculateRailwayRent(game, owner);
else
    rent = calculateUtilityRent(game, owner, diceValue);

payMoney(game, playerIndex, rent);
receiveMoney(game, owner, rent);

formatLKR(rent, rentBuf);

```

```
printf("\n%s landed on %s.\n",
      game[0].players[playerIndex].name,
      game[0].board[pos].name);

printf("Rent Paid : LKR %s.\n", rentBuf);

printf("Owner : %s.\n", game[0].players[owner].name);
}
```